nomiso

Request for Information (RFI)

Strategic Development Partnership for Intelli-Q
AI-Powered SDLC & Engineering Intelligence Platform

© 2026 Nomiso. All rights reserved.

Table of Contents

1. Introduction & Background 3
2. Current Intelli-Q Capabilities (Already Available) 3
    2.1 Requirements & User Story Intelligence 3
    2.2 Knowledge Intelligence Layer 4
    2.3 Test Automation & Engineering Support 5
3. Scope of Development – Required Capabilities 5
Section A : UI/UX & Workflow Layer Enhancements 6
    A1: SBP (Solution Blueprint) Generation Screens 6
    A2: Unit Test Case Generation Screens 7
    A3: Test Script Generation Interface 7
    A4: Test Framework Migration Engine 8
Section B : UI/UX & Workflow Layer Enhancements 9
    B1: HLD/LLD Generation Engine 9
    B2: Design Assistance Engine 9
    B3: Automated Code Generation 10
    B4: Code Reverse Engineering 10
    B5: Advanced Code Impact Analysis 11
    B6: Automated Code Review 11
    B7: Code Assessment Integration 12
4. Technical & Architecture Expectations 12
5. Delivery & Governance Model 12
6. Evaluation Criteria 13
7. Vendor Response Requirements 13
8. Strategic Vision 13

© 2026 Nomiso. All rights reserved.

1. Introduction & Background

Nomiso invites qualified technology partners to participate in this Request for Information (RFI) to support the next phase of development of Intelli-Q, Nomiso's AI-powered SDLC and Test Intelligence product.

Intelli-Q is designed to embed intelligence across the software lifecycle—spanning requirements engineering, test case generation, automation scripting, knowledge management, and engineering assistance. The platform leverages LLM orchestration, Retrieval-Augmented Generation (RAG), structured validation layers, and enterprise integrations to accelerate SDLC timelines while improving quality and governance.

As part of our strategic roadmap, Nomiso is expanding Intelli-Q into a broader AI-driven Engineering Intelligence Platform, covering advanced design generation, code intelligence, impact analysis, and automated review capabilities.

Through this RFI, Nomiso seeks a long-term co-innovation partner capable of delivering scalable, enterprise-grade AI features with strong UI/UX maturity, architecture depth, and product engineering discipline.

2. Current Intelli-Q Capabilities (Already Available)

The following modules are currently functional within Intelli-Q. Partners should demonstrate their ability to extend and enhance these capabilities while maintaining architectural consistency.

2.1 Requirements & User Story Intelligence

Context

Intelli-Q integrates with JIRA and other requirement repositories to ingest, parse, validate, and enhance user stories and acceptance criteria.

Current Capabilities

- Knowledge Management
  - Upload Doc/ PDF
  - View document list
  - View document details / Chunk details
  - Search document
  - Delete document
- Job Management

© 2026 Nomiso. All rights reserved.

    - View Job List
    - Search Job List
    - View Job details
    - Close/ Cancel Job
- User Story Enhancer
    - User Story Reader (JIRA Integration)
    - User Story Reader (Document/ PDF uploads)
    - User Story Parser
    - User Story Writer (JIRA)
    - User Story Writer (Document/ PDF downloads)
    - Requirement Validation Engine
- Functional Test Case Generation
    - User Story Reader (JIRA Integration)
    - User Story Reader (Document/ PDF uploads)
    - Test Case Planner
    - Test Case Generator
    - Test Case Reviewer
    - Test Case Writer (JIRA)
    - Test Case Writer (Document / PDF downloads)
    - Search Test Cases
    - View Test Case List
    - View Test Case Details

Strategic Intent

This layer ensures structured transformation of requirements into test-ready artifacts, reducing ambiguity and ensuring traceability (Story → Acceptance Criteria → Test Case).

2.2 Knowledge Intelligence Layer

Context

Intelli-Q operates using a vectorized Knowledge Base architecture that stores structured enterprise artifacts for contextual AI generation.

Current Capabilities

- Knowledge Base Creation (RAG-based)
- Knowledge Base Management UI
- Chunking and Embedding of Documents
- Context Retrieval Optimization

Strategic Intent

© 2026 Nomiso. All rights reserved.

This ensures AI outputs are grounded in enterprise-specific knowledge rather than generic generation, improving accuracy and reducing hallucination risk.

2.3 Test Automation & Engineering Support

Context

Intelli-Q accelerates automation and unit testing by generating executable scripts aligned with functional intent.

Current Capabilities

- Unit Test Case Generation
- Selenium Script Generation
- Appium Script Generation
- VS Code Extension
- JIRA Plugin Integration
- SBP (Solution Blueprint) Creation

Strategic Intent

This layer reduces manual scripting effort, accelerates automation coverage, and integrates into CI/CD workflows

3. Scope of Development – Required Capabilities

The following modules require comprehensive design, development, enhancement, and integration as part of the next phase of Intelli-Q's product evolution.

This scope includes:

- End-to-end feature development, covering frontend (UI/UX), backend services, AI agent, RAG implementation, Agentic Workflow orchestration layers, and integration components.
- Backend engineering and service-layer development for all new modules, including API design, data modeling, orchestration workflows, validation layers, logging, and security controls.
- Extension and optimization of the existing Intelli-Q architecture to support scalability, modularity, and enterprise-grade performance.

Wherever existing APIs, microservices, orchestration pipelines, or integration components are already available within Intelli-Q, vendors are expected to:

© 2026 Nomiso. All rights reserved.

- Review and reuse existing assets to maintain architectural consistency
- Extend or enhance current services rather than duplicating functionality.
- Ensure backward compatibility and minimal disruption to existing modules.
- Align new development with the current RAG-based knowledge architecture, validation engines, and integration frameworks.

The development partner will be responsible for:

- Ensuring seamless integration with current Intelli-Q capabilities (JIRA integration, Knowledge Base layer, test generation engines, etc.).
- Maintaining standardized coding practices, API governance, and documentation.
- Designing modular, extensible backend services that support future roadmap expansion.
- Implementing appropriate security, audit logging, RBAC, and performance monitoring mechanisms.

This phase is not limited to UI enablement; it requires deep backend engineering, AI agent, RAG implementation, Agentic Workflow **orchestration integration, and enterprise-grade system design to ensure Intelli-Q evolves into a scalable Engineering Intelligence Platform.**

Section A : UI/UX & Workflow Layer Enhancements

A1: SBP (Solution Blueprint) Generation Screens

Context

Intelli-Q currently supports SBP (Solution Blueprint) creation; however, the user interface and workflow experience are not fully productized. As SBP becomes foundational to design traceability, architecture validation, and downstream automation, a structured and guided UI is required.

Feature Description

The SBP Generation module should provide a structured workflow that transforms requirements and user stories into formalized solution blueprints. This includes:

- Functional architecture breakdown
- Technical component mapping
- Integration touchpoints
- Data flow visualization
- Traceability to user stories and acceptance criteria

The UI must support:

© 2026 Nomiso. All rights reserved.

- Guided multi-step blueprint creation
- AI-assisted content suggestions and refinement
- Editable schema sections (Functional, Technical, Integration)
- Version control and comparison
- Export capabilities (PDF, DOCX, API formats)
- Audit and approval workflows

This module will act as a bridge between requirements intelligence and engineering execution.

A2: Unit Test Case Generation Screens

Context

While Intelli-Q supports unit test generation, there is no dedicated interactive review interface that allows engineers to validate, refine, and manage generated tests at scale.

Feature Description

The Unit Test Generation module should provide a developer-centric interface that enables:

- Code ingestion (file upload, repository connection, or inline paste)
- AI-driven unit test generation
- Code coverage estimation preview
- Editable test case refinement
- Framework compatibility selection (e.g., NUnit, xUnit, MSTest)
- CI/CD trigger capability

The system should support:

- Expandability to additional languages (Java, Python, etc.)
- Test case traceability to methods/classes
- Coverage gap highlighting
- Export and repository push integration

This feature enhances developer productivity while maintaining engineering control.

A3: Test Script Generation Interface

Context

Intelli-Q generates Selenium and Appium scripts; however, users require structured visualization, validation, and framework selection options within an integrated UI.

Feature Description

© 2026 Nomiso. All rights reserved.

The Test Script Generation module should provide:

- Framework selection options (Selenium/Appium/Hybrid)
- Script preview with syntax highlighting
- Editable AI output interface
- Refactoring suggestions
- Environment configuration mapping
- Regression tagging and categorization

Additional capabilities:

- Cross-browser/device selection panel
- Script modularization recommendations
- Migration assistance (legacy to modern frameworks)

This module should support automation scalability and reduce script brittleness.

A4: Test Framework Migration Engine

Context

Many enterprise clients use legacy or inconsistent automation frameworks. Migration to standardized, maintainable frameworks is complex and risky.

Feature Description

This module should:

- Analyze existing test frameworks
- Identify deprecated dependencies
- Provide AI-driven migration suggestions
- Transform legacy scripts into standardized frameworks
- Assess migration risk and impact

Key components:

- Code parsing engine
- Dependency mapping
- Framework comparison matrix
- Automated transformation assistant
- Risk scoring and rollback guidance

This capability reduces modernization effort and improves maintainability.

© 2026 Nomiso. All rights reserved.

Section B : UI/UX & Workflow Layer Enhancements

B1: HLD/LLD Generation Engine

Context

Architecture documentation is often manually created and inconsistently maintained. Intelli-Q should automate generation of High-Level Design (HLD) and Low-Level Design (LLD) artifacts from requirements, APIs, and code.

Feature Description

The engine should:

- Generate structured HLD/LLD documentation
- Suggest architecture diagrams
- Map integrations and APIs
- Highlight dependencies
- Maintain traceability to user stories

Expected outputs:

- Component diagrams
- Sequence diagrams
- Data flow diagrams
- Infrastructure topology

This reduces manual documentation effort and improves design consistency.

B2: Design Assistance Engine

Context

Engineering teams often lack structured guidance during early-stage design decisions. An AI-powered advisory layer can improve architecture robustness.

Feature Description

The Design Assistance Engine should:

- Recommend architectural patterns (microservices, layered, event-driven)
- Suggest scalability considerations
- Provide security and compliance guidance
- Offer API best practice suggestions
- Flag potential performance bottlenecks

© 2026 Nomiso. All rights reserved.

It should function as a contextual AI co-pilot during design drafting.

B3: Automated Code Generation

Context

To reduce SDLC cycle times, Intelli-Q aims to generate production-grade code directly from validated requirements and design artifacts.

Feature Description:

This feature should:

- Generate modular, standards-compliant code
- Follow configurable coding templates
- Include documentation comments
- Support multi-language extensibility
- Align with enterprise coding standards

Safeguards must include:

- Validation layer
- Hallucination mitigation
- Secure coding compliance checks

This accelerates development while maintaining governance.

B4: Code Reverse Engineering

Context

Many enterprises operate legacy codebases with poor documentation. Reverse engineering enables better understanding and modernization.

Feature Description

The module should:

- Parse large codebases
- Generate architecture maps
- Extract APIs and integration points
- Identify technical debt
- Build dependency graphs

Output formats may include:

© 2026 Nomiso. All rights reserved.

- Graph visualizations
- Structured documentation
- Refactoring suggestions

B5: Advanced Code Impact Analysis

Context

Changes in code often have cascading effects across modules, tests, and integrations. Current impact analysis is manual and incomplete.

Feature Description

This engine should:

- Detect code deltas
- Map dependency impact
- Predict regression scope
- Suggest targeted test execution
- Provide risk scoring

This directly enables regression optimization and release risk reduction.

B6: Automated Code Review

Context

Code reviews are time-intensive and often inconsistent. AI-assisted reviews can standardize quality enforcement.

Feature Description

The module should:

- Analyze code for performance issues
- Detect security vulnerabilities
- Enforce coding standards
- Provide inline contextual suggestions
- Integrate with PR workflows

It must include:

- False positive mitigation
- Policy configuration layer
- Human-in-the-loop validation

© 2026 Nomiso. All rights reserved.

B7: Code Assessment Integration

Context

Enterprises use multiple code quality tools (e.g., static analysis, security scanners). Intelli-Q must aggregate and contextualize insights.

Feature Description

This capability should:

- Integrate via APIs with external tools
- Consolidate metrics into unified dashboards
- Correlate assessment findings with user stories and tests
- Provide actionable insights

This ensures centralized SDLC intelligence.

4. Technical & Architecture Expectations

Vendors must demonstrate:

- LLM orchestration expertise
- RAG architecture experience
- Microservices-based design
- Scalable cloud-native implementation
- Enterprise security (RBAC, encryption, audit logging)
- Performance optimization for high-volume workloads
- Human-in-the-loop validation frameworks

5. Delivery & Governance Model

Vendors must provide:

- Proposed engagement model
- Team composition
- Agile delivery methodology
- Timeline assumptions
- Risk mitigation strategy
- IP protection approach
- Data privacy compliance

© 2026 Nomiso. All rights reserved.

6. Evaluation Criteria

Partners will be evaluated on:

- AI & SDLC domain depth
- UI/UX maturity
- Developer tool experience
- Architecture scalability
- Enterprise readiness
- Security compliance
- Long-term innovation capability

7. Vendor Response Requirements

Vendors must submit:

- Company profile
- Relevant AI/LLM case studies
- Architecture samples
- Delivery model proposal
- Indicative commercial structure
- Client references

8. Strategic Vision

Nomiso intends to evolve Intelli-Q into a comprehensive AI-powered Engineering Intelligence Ecosystem—reducing SDLC timelines, improving productivity, enabling predictive engineering decisions, and embedding intelligence across the software lifecycle.

We seek a partner capable of co-building this vision at enterprise scale.

© 2026 Nomiso. All rights reserved.